Network I/O Primer

`socket`、blocking / nonblocking I/O、`select/poll/epoll`、socket buffer、`go-runtime-primer` の `netpoll` を、Linux x86-64 の user/kernel 境界として整理する学習用ノートです。`syscalls-process-primer` で見た syscall / sleep / wakeup の感覚を network I/O へ伸ばし、Go の goroutine が I/O 待ちでどう止まり、どう再開するのかへ戻れるように構成しています。

> **編集注記**
> この文書は `socket` API や Linux source の逐語訳ではありません。`socket` を作る、待つ、再開する、複数 connection をさばく、runtime とつなぐ、という学習の主線が見えやすい順に再構成した版です。TCP/IP 全史や NIC driver の網羅ではなく、「network I/O は待ち合わせの問題でもある」という感覚を掴むことを優先しています。

目次

Contents

はじめに 2
- この教材の主題 3
- なぜ今 network I/O を学ぶのか 3
- 対象と範囲 4
- 最初に持っておく直感 4
- 参照する一次資料 4
- 各章の見取り図 5

`socket` と blocking I/O 5
- `socket` は何者か 5
- listening socket と connected socket 6
- 最小の blocking echo server 6
- 何が block しているのか 7
- `read` の戻り値の見方 7
- client 側はどう見えるか 7
- shell から動かしてみる最小例 8
- Go で見ると何が同じか 8
- 1 client ずつ処理する構造の限界 8
- この章で作りたい感覚 9

nonblocking I/O と readiness 9
- なぜ nonblocking が必要か 9
- `O_NONBLOCK` 9
- `read` が返す「今は無理」 10
- `write` と partial write 10
- `readable` と `writable` の意味 11
- receive buffer / send buffer と backpressure 11
- timeout をどう考えるか 11
- nonblocking は busy loop のことではない 11
- Go から見ると何が起きているか 12

この章で作りたい感覚 12
select / poll / epoll 12
1 本ずつ見るのではなく、まとめて待つ 12
select 13
poll 13
epoll 13
最小の epoll event loop 13
1 event = 1 request 完了ではない 14
level-triggered と edge-triggered 14
accept loop も drain する 15
EPOLLOUT は必要なときだけ 15
select / poll / epoll の使い分けの感覚 15
Go runtime への前振り 16
kernel 側の待ち合わせと runtime への橋 16
listen backlog の最小像 16
socket buffer と data の流れ 16
backpressure はどこから来るか 17
loopback は特別か 17
TCP と UDP の違いを最小限だけ 17
data 到着から wakeup までの最小像 17
sleep/wakeup と scheduler 18
Go runtime の netpoll 18
C の event loop と Go の goroutine-per-connection 18
Go の最小 echo server をもう一度見る 19
readiness ベースの待ち合わせの限界と次の話題 19
この章で作りたい感覚 20
付録と次の一歩 20
章ごとの復習ポイント 20
観測コマンドの導線 20
小さな観測課題 21
用語小事典 21
よくあるつまずき 21
readable なら 1 request 全体を読めると思ってしまう 21
nonblocking は busy loop だと思ってしまう 21
epoll があれば state machine は不要だと思ってしまう 21
Go では epoll を意識しなくてよいと思ってしまう 22
writable なら全部送れると思ってしまう 22
短い演習案 22
syscalls-process-primer と go-runtime-primer へ戻る読み方 22
次に読むなら 22
参考資料 23
おわりに 23

はじめに

この文書は、Linux x86-64 上の network I/O を「socket を作る」「待つ」「再開する」という一本の線で整理するためのノートです。ネットワークを学ぶと聞くと、Ethernet frame、IP

header、TCP header、routing table の図から入りたくなります。しかし user space の program から見ると、最初にぶつかる問いはもっと手前にあります。なぜ `accept` は待つのか。なぜ `read` は止まるのか。なぜ 1 本の thread で大量の socket を扱えるのか。なぜ Go では goroutine を connection ごとに作っても破綻しにくいのか。本書は、その問いに答えるための導入です。

`syscalls-process-primer` では、syscall、sleep、wakeup、scheduler の最小像を扱いました。一方で、`epoll` や async I/O の話は意図的に外しています。`go-runtime-primer` では `netpoll` が重要だと触れましたが、OS 側の network I/O の足場は深追いしていません。本書の役割は、その間を埋めることです。network I/O を「プロトコルの話」だけでなく、「どの実行主体が、いつ眠り、どの条件で起きるか」という low-level の話として見えるようにします。

この教材の主題

本書の主線は次の 5 点です。

1. `socket` が file descriptor としてどう見えるか
2. blocking I/O と nonblocking I/O の違いは何か
3. `select` / `poll` / `epoll` が何を待ち合わせているのか
4. socket buffer、backlog、sleep/wakeup がどうつながるのか
5. その上に Go runtime の `netpoll` がどう乗るのか

この順序を取る理由は、いきなり `epoll` や `netpoll` へ飛ぶと、「何を登録して、何が ready になっているのか」が曖昧なままになるからです。まず `socket` と blocking I/O の手触りを作り、そのあとで nonblocking と readiness へ進みます。最後に user-space runtime の都合へ戻ると、goroutine ごとの見え方と kernel 側の待ち合わせをつなげて理解しやすくなります。

この章の到達点

本書を読み終えるころには、少なくとも次を説明できる状態を目指します。

- listening socket と connected socket の違い
- `O_NONBLOCK` と `EAGAIN` が意味すること
- `select`、`poll`、`epoll` の役割差分
- `readable` / `writable` が「完全に終わった」を意味しない理由
- Go の network I/O が goroutine をどう止め、どう再開するかの概観

なぜ今 network I/O を学ぶのか

低レイヤ学習では、CPU、メモリ、実行形式、syscall、runtime までは比較的一本の線に見えます。しかし network は別の島に見えやすいです。packet の図が多くなり、急に protocol 専門書の雰囲気になるからです。もちろん protocol は重要です。ただ、program を書く側が最初に知りたいのは、packet の bit 配置そのものよりも、「いまこの `read` がなぜ返らないのか」「複数 client をどう待つのか」「goroutine は本当に thread を占有していないのか」といった問いであることが多いです。

その意味で、network I/O は `syscalls-process-primer` の自然な次です。`read` と `write` を file に対して使うだけなら、blocking の感覚だけでもある程度進めます。しかし socket が相手になると、相手の速度、buffer の空き、接続待ち、再送、輻輳、wake up の契機が絡みます。ここで初めて、「I/O は単なる関数呼び出しではなく、待ち合わせを含む同期の問題だ」という実感が強くなります。

さらに Go runtime を読むときにも、この直感は効きます。goroutine が `net.Conn.Read` で止まったとき、本当に OS thread がそのまま寝ているのか。`netpoll` は何を poll しているのか。runtime は kernel の readiness 通知をどう user-space scheduler へ戻しているのか。この問いは、`epoll` と nonblocking socket の感覚なしには少し掴みにくいです。

対象と範囲

対象は Linux x86-64 に固定します。`socket` API 自体は POSIX 風ですが、`epoll` は Linux 固有ですし、Go runtime の network poller も OS ごとに backend が違います。初学段階では一般論を広げるより、1 つの具体環境で筋を通したほうが理解しやすいです。したがって本書では、Linux の `socket`、`fcntl(O_NONBLOCK)`、`select` / `poll` / `epoll`、`tcp(7)` / `udp(7)`、Go runtime の `netpoll` を主な足場にします。

一方で、初版では次を主題にしません。

- Ethernet、ARP、IP、TCP の全 header を bit 単位で読むこと
- kernel の TCP 実装 source を最後まで追い切ること
- NIC driver、DMA、割り込み処理、NAPI の細部
- TLS、HTTP/2、QUIC といった上位 protocol
- `io_uring` や completion-based I/O の深掘り

これらはすべて価値がありますが、最初の一冊で全部を同時に扱うと中心線がぼやけます。本書の初版は「user space の code から見た network I/O」と「それを支える待ち合わせ」の理解を優先します。

最初に持っておく直感

network I/O を理解するとき、最初に持っておくと楽な直感が 3 つあります。

第一に、socket も file descriptor の 1 種だということです。つまり user space から見れば、`read`、`write`、`close`、`fcntl`、`poll` のような既存の I/O API の枠内でかなりの部分が扱えます。もちろん内部実装は file と違いますが、入口の形が揃っていることは大きいです。

第二に、`readable` や `writable` は「全仕事が終わった」という意味ではないことです。`readable` は「少なくとも今なら 1 回は進めそう」という合図です。1 byte だけ読める場合もありますし、EOF が見える場合もあります。`writable` も「大きな payload を全部送り切れる」とは限りません。この直感がないと、event loop や `epoll` の説明が少し不思議に見えます。

第三に、network I/O は相手都合の待ち合わせだということです。file の `read` なら、ディスク cache や page cache の都合はあっても、概念上は「そこにある data を読む」感覚を持ちやすいです。socket は違います。相手がまだ送っていないかもしれないし、相手の受信 buffer が詰まっているかもしれないし、3-way handshake が終わっていないかもしれません。そのため、sleep と wakeup の話が中心へ出てきます。

> **注意**
> 本書では説明のために単純化したモデルを使います。たとえば `listen` backlog も最初は「`accept` 待ちの queue」として説明します。しかし実際の Linux TCP stack は handshake 中の queue や tuning parameter を含み、もう少し複雑です。ここでは細部の完全再現よりも、まず役割の違いを掴むことを優先します。

参照する一次資料

本書では、次の資料を主に参照します。

- `socket(2)`, `bind(2)`, `listen(2)`, `accept(2)`, `connect(2)`
- `read(2)`, `write(2)`, `fcntl(2)`
- `select(2)`, `poll(2)`, `epoll(7)`
- `tcp(7)`, `udp(7)`, `socket(7)`
- `strace`, `ss`, `tcpdump` の manual
- Go runtime の `netpoll` 関連 source と `go tool trace`

これらをそのまま読むと、API manual、kernel 都合、runtime 都合が散らばって見えます。本書ではそれらを「1 本の connection がどのように待たれ、どのように再開されるか」という軸で束ねます。

> **編集注記**
> network の学習で最初から packet header へ入りたくなる気持ちは自然です。ただ、socket を使う program の側から見ると、先に理解すべきなのは「いつ block するか」「誰が wake up するか」です。本書が `socket` と I/O の話から始めるのはそのためです。

各章の見取り図

本書は次の順で進みます。

1. まず `socket` と blocking I/O の最小像を作る
2. 次に nonblocking I/O と readiness の意味を整理する
3. その上で `select` / `poll` / `epoll` を比較し、event loop の形を見る
4. さらに backlog、buffer、sleep/wakeup、TCP/UDP の違いを最小限だけ押さえる
5. 最後に Go runtime の `netpoll` へ戻り、goroutine の待ち方とつなぐ

この順序には理由があります。`epoll` は魅力的ですが、その前に「何が block し、何を待っているのか」が見えていないと、単なる API 暗記になりやすいからです。次章ではその土台として、blocking socket を使った最小の server/client の流れから始めます。

`socket` と blocking I/O

この章では、まず network I/O を最も素朴な形で見ます。1 本の listening socket を作り、client が来るまで `accept` で待ち、接続後は `read` / `write` でやり取りする、という基本形です。最初から nonblocking や `epoll` を出さない理由は、何が block するのかを自分の目で確認したほうが、後の抽象化がずっと楽になるからです。

> **この章の到達点**
> この章では次を押さえます。
>
> - socket が file descriptor として見えること
> - listening socket と connected socket の役割の違い
> - blocking `accept` / `read` / `write` が何を待っているか
> - 1 client ずつ処理する server の限界

`socket` は何者か

user space から見ると、socket は file descriptor を返す特別な open 操作です。`open` の代わりに `socket(domain, type, protocol)` を呼び、その返り値として整数 fd を受け取ります。以後、その fd に対して `read`、`write`、`close`、`fcntl`、`poll` などを使えます。つまり、program の入口では「I/O 対象が file か socket か」はかなり統一された形に見えます。

もちろん内部実装は違います。通常のファイルなら page cache や inode が中心ですし、socket なら protocol の状態、送受信 buffer、接続待ち queue が中心です。ただ、fd という共通のハンドルを介して見えるため、kernel は「待てるもの」「読めるもの」「書けるもの」をある程度統一的に扱えます。後で `select` や `epoll` が socket だけでなく pipe や eventfd と一緒に扱えるのも、この統一のおかげです。

listening socket と connected socket

TCP server を考えると、最初に作るのは listening socket です。これは「接続要求を受け取る入口」であり、data 本体をやり取りする socket ではありません。典型的には次の順で準備します。

1. `socket(AF_INET, SOCK_STREAM, 0)` で TCP socket を作る
2. `bind` で local address と port を結び付ける
3. `listen` で passive open の状態へ入る
4. `accept` で新しい接続を取り出す

ここで重要なのは、`accept` が返す fd は listening socket とは別物だということです。listening socket は引き続き新しい接続要求を受けるために残り、実際の送受信は `accept` が返した connected socket で行います。この二段構えを最初に理解しておくと、server の構造がかなり見やすくなります。

最小の blocking echo server

まずは C で最小の blocking TCP echo server を見ます。実際の production code なら error handling、`setsockopt(SO_REUSEADDR)`、signal 対応などが必要ですが、ここでは主線だけに絞ります。

```
int lfd = socket(AF_INET, SOCK_STREAM, 0);

struct sockaddr_in addr = {
    .sin_family = AF_INET,
    .sin_port = htons(8080),
    .sin_addr.s_addr = htonl(INADDR_LOOPBACK),
};

bind(lfd, (struct sockaddr *)&addr, sizeof(addr));
listen(lfd, 128);

for (;;) {
    int cfd = accept(lfd, NULL, NULL);
    char buf[4096];
    ssize_t n = read(cfd, buf, sizeof(buf));
    if (n > 0) {
        write(cfd, buf, (size_t)n);
    }
    close(cfd);
}
```

この code はかなり単純ですが、network I/O の骨格が全部入っています。`accept` は新しい client が来るまで待ちます。接続が成立すると connected socket `cfd` が返り、その socket から `read` で data を受け、`write` で返します。client が何も送ってこなければ `read` は待ちます。つまり、block する地点が 2 箇所あります。

- 接続待ちの `accept`

- data 待ちの `read`

`write` も状況によっては block します。相手の受信側が遅かったり、送信 buffer に空きがなかったりすると、今すぐ全部は書けません。blocking socket では、その空きが出るまで caller を眠らせることがあります。

> **編集注記**
> 「blocking」と聞くと CPU が忙しく回り続ける印象を持つことがありますが、普通は逆です。kernel はその thread を wait queue へ置き、条件が満たされるまで sleep させます。busy loop と blocking wait は全く別物です。

何が block しているのか

`accept` が block しているとき、server は「新しい接続を受け取れる状態になるまで待っている」と言えます。TCP の細部まで踏み込まなくても、少なくとも user space からは「いまは取り出すべき新規接続がない」状態として見えます。新しい接続が queue へ入り、`accept` 可能になると kernel は sleeping thread を起こし、`accept` が connected socket を返します。

`read` が block しているときは、「その socket の受信 buffer に返せる data がまだない」という理解で十分です。相手が 1 byte 送ってきただけでも `read` は返るかもしれませんし、相手が close したなら 0 を返します。つまり `read` は「完全な application message」を待っているのではなく、「今返せる kernel buffer の状態変化」を待っています。

`write` も同様です。相手がまだ読んでいないため送信 buffer が詰まっているなら、今すぐには進めません。blocking socket では、その空きが出るまで caller が眠ります。ここに network I/O の相手都合が見えます。file への `write` よりも、「向こうの速度」が前面に出てくるわけです。

`read` の戻り値の見方

socket の `read` は少なくとも次の 3 パターンで見ます。

- `n > 0` data が `n` byte 読めた
- `n == 0` peer が orderly shutdown し、EOF が見えた
- `n == -1` error。blocking socket なら通常は即時 error、nonblocking なら後で見る `EAGAIN` もここに入る

ここで重要なのは、`n > 0` が「要求した長さを全部読めた」を意味しないことです。たとえば 4096 byte 読みたくても、今 buffer に 100 byte しかなければ 100 だけ返ることがあります。TCP は byte stream なので、message 境界を守ってくれません。application protocol 側で長さや区切りを決める必要があります。

client 側はどう見えるか

client は `connect` を使います。最小形は次のようなものです。

```
int fd = socket(AF_INET, SOCK_STREAM, 0);
connect(fd, (struct sockaddr *)&addr, sizeof(addr));
write(fd, "hello\n", 6);
read(fd, buf, sizeof(buf));
close(fd);
```

blocking connect も待つ可能性があります。相手に届くまで、あるいは失敗が確定するまで caller は返ってきません。つまり TCP client でも、接続確立の時点からすでに「相手都合の待ち合わせ」が始まっています。

shell から動かしてみる最小例

動作確認の雰囲気だけなら、nc と組み合わせると掴みやすいです。

```
$ gcc -O2 server.c -o server
$ ./server

別 terminal:
$ nc 127.0.0.1 8080
hello
hello
```

この段階では、1 client がつながって何も送らないだけで server 側の read は止まります。code が 1 本の実行線しか持っていないため、別の client を同時に扱えません。ここが blocking server の最初の限界です。

Go で見ると何が同じか

Go で同じことを書くと、見た目は少し変わります。

```go
ln, _ := net.Listen("tcp", "127.0.0.1:8080")
for {
    conn, _ := ln.Accept()
    go func(c net.Conn) {
        defer c.Close()
        io.Copy(c, c)
    }(conn)
}
```

この code は goroutine を使っているので C の最小例より柔らかく見えます。しかし骨格は同じです。Listen は listening socket を作り、Accept は新しい connected socket を受け取り、Read / Write 相当を通じて data をやり取りします。違うのは、「複数 connection をどう同時にさばくか」の部分を runtime がかなり肩代わりしてくれることです。

本章の段階では、Go の見た目に引っ張られ過ぎないことが大切です。下にある kernel 側の待ち合わせは消えていません。user code が blocking API に見えていても、その裏では nonblocking 化や poller 連携が使われます。この話は後半で戻ります。

1 client ずつ処理する構造の限界

最初の C echo server は、理解の入口としては優れていますが、server としてはすぐ限界にぶつかります。1 client が接続したまま黙っていると、その read が返るまで次の接続処理へ進めません。1 thread あたり 1 本の blocking 実行線しか持っていないからです。

もちろん対処法はあります。connection ごとに thread や process を作る、あるいは先に socket を nonblocking にし、複数 fd をまとめて待つ、といった方向です。前者は分かりやすいですが、connection 数が増えると thread 数や memory 使用量が重くなります。後者が select / poll / epoll へつながる道です。

> **注意**
> accept が 1 回返ったからといって、その connection で大量の data をすぐ読めるとは限り

ません。また `write` が 1 回成功したからといって、application が送りたい全 message を出し切れたとは限りません。network I/O は「少し進む」を積み重ねる世界です。

この章で作りたい感覚

この章で大事なのは、socket の全 API を暗記することではありません。むしろ次の感覚を持つことです。

- listening socket と connected socket は別の役割を持つ
- blocking call は「条件が整うまで sleep する」
- network I/O は相手都合で止まりやすい
- `read` / `write` の 1 回の成功は、完全な application-level 進捗を保証しない

この感覚があれば、次章の nonblocking I/O と readiness はかなり自然に見えます。`block しない` とは何か。`今は無理` をどう受け取るのか。`readable` / `writable` は何を知らせるのか。そこを次に整理します。

nonblocking I/O と readiness

blocking socket は分かりやすい反面、1 本の実行線を簡単に止めてしまいます。少数 connection ならそれでも動きますが、大量 connection を 1 thread で扱いたくなると、blocking の素朴さが重荷になります。この章では、その重荷を外すために nonblocking I/O と readiness の意味を整理します。

この章の到達点

この章では次を押さえます。

- `O_NONBLOCK` を付けると何が変わるか
- `EAGAIN` / `EWOULDBLOCK` をどう読むか
- partial read / partial write がなぜ普通に起きるか
- `readable` / `writable` が何を保証し、何を保証しないか
- backpressure と timeout をどう考えるか

なぜ nonblocking が必要か

1 本の thread で大量の socket を扱いたいとき、最も困るのは「どの socket が先に進むかを自分で選べない」ことです。blocking `read` を 1 本呼ぶと、その socket に data が来るまで thread 全体が止まります。他の socket が ready でも、その thread は見に行けません。

ここで欲しいのは、「今すぐ進める socket だけを少しずつ進めたい」という制御です。そのための前提が nonblocking I/O です。socket を nonblocking にすると、`read` や `write` は「待てないなら今は返る」ようになります。thread を眠らせる代わりに、caller へ「今は無理」という結果を返して制御を戻すわけです。

O_NONBLOCK

Linux では `fcntl` で `O_NONBLOCK` を付けるのが典型です。

```
int flags = fcntl(fd, F_GETFL, 0);
fcntl(fd, F_SETFL, flags | O_NONBLOCK);
```

これ以後、その fd に対する `read`、`write`、`accept`、`connect` などは、待てないときに sleep しません。代わりに `-1` を返し、`errno` に `EAGAIN` あるいは `EWOULDBLOCK` を入れます。実装上は両者を同じ意味で扱うことが多いです。

ここで重要なのは、`EAGAIN` を「壊れた error」だと思わないことです。これは多くの場合、「今は条件が足りないので、待てる仕組みに戻してほしい」という合図です。つまり nonblocking I/O の世界では、`EAGAIN` は失敗というより制御フローの一部です。

> **編集注記**
> blocking I/O では kernel が caller の代わりに wait してくれます。nonblocking I/O ではその wait を user space へ戻します。どちらが偉いという話ではなく、「どこで待ち合わせを組み立てるか」が違うだけです。

`read` が返す「今は無理」

nonblocking socket で `read` すると、少なくとも次の 4 通りを意識します。

- `n > 0` 読めた data がある
- `n == 0` peer が close し、EOF が見えた
- `n == -1 && errno == EAGAIN` 今は読める data がない
- `n == -1 && errno` がそれ以外 実際の error

blocking I/O と違い、`EAGAIN` が普通に出てきます。ここで「何度も `read` を呼び続ける」のは良くありません。data が来るまで busy loop してしまうからです。正しい流れは、「今は無理」と分かったら poller へ登録し、readable になるまで別の仕事をする、です。

`write` と partial write

`write` でも同じことが起きます。nonblocking socket に大量の data を書こうとしても、送信 buffer に空きがなければ全部は入りません。すると `write` は短く返るか、あるいは `EAGAIN` を返します。したがって nonblocking の送信処理は「1 回 `write` して終わり」ではなく、送れた分だけ進め、残りを保持し、writable 通知で再開する構造になります。

典型的には次のような loop です。

```
while (off < len) {
    ssize_t n = write(fd, buf + off, len - off);
    if (n > 0) {
        off += (size_t)n;
        continue;
    }
    if (n == -1 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
        remember_unsent_bytes(fd, buf + off, len - off);
        arm_writable_interest(fd);
        break;
    }
    handle_fatal_write_error(fd);
    break;
}
```

この pattern が必要なのは、TCP が byte stream であり、kernel が「どこまで送れたか」しか保証しないからです。message 境界は application 側で管理する必要があります。

`readable` と `writable` の意味

ここで readiness という言葉を導入します。`readable` は「今 `read` を試せば、少なくとも 1 回は block せずに進めそう」という意味です。`writable` は「今 `write` を試せば、少なくとも少しは block せずに進めそう」という意味です。

この定義には含意があります。

- `readable` は「1 message 全体が揃った」を保証しない
- `writable` は「送りたい data を全部出せる」を保証しない
- readiness は completion ではなく、try する権利に近い

この感覚が非常に重要です。たとえば HTTP request 全体を 1 回の `read` で取り切れるとは限りません。逆に `readable` なのに `read` してみたら EOF だった、ということもあります。kernel が知らせているのは、socket buffer の状態変化です。

receive buffer / send buffer と backpressure

socket には送受信 buffer があります。受信側では、network から届いた data が一時的に receive buffer へ入り、user space の `read` で取り出されます。送信側では、user space が `write` した data が send buffer へ入り、network へ送り出されます。

ここで backpressure という現象が起きます。相手がゆっくり読むと、こちらの send buffer はなかなか空きません。すると `write` は進みにくくなります。逆にこちらが読まないと receive buffer が詰まり、相手側の送信も進みにくくなります。つまり network I/O では、読む速さと書く速さが連鎖します。

backpressure を理解すると、「なぜ writable notification が必要なのか」「なぜ送信 queue を user space に持つ必要があるのか」が見えてきます。server は単に socket が readable かどうかだけでなく、未送信 data の有無も管理しなければいけません。

timeout をどう考えるか

blocking I/O では、単純に長く待つ code を書きやすいです。しかし nonblocking I/O では、待ちそのものを user space が組み立てます。したがって timeout も user space 側の責務へ寄ってきます。

最も単純なのは poll 系 API の timeout を使うことです。`poll` や `epoll_wait` は待ち時間を指定できます。ただし本当に欲しいのは、API 呼び出し 1 回の timeout ではなく、「この connection は 30 秒 idle なら切る」「この request は 5 秒以内に header を読みたい」といった higher-level な timeout です。したがって event loop を書くときは、fd readiness と timer を一緒に管理することが多くなります。

nonblocking は busy loop のことではない

ここで誤解しやすい点を強調します。nonblocking にしたからといって、「ひたすら `read` を叩き続ける」のは正しくありません。それでは CPU を無駄に回すだけです。nonblocking I/O の本質は、「待ちを kernel から user space に戻し、その待ちをまとめて管理する」ことです。実際に待つ場所は消えません。ただ位置が変わるだけです。

したがって典型的な流れは次です。

1. socket を nonblocking にする
2. `read` / `write` / `accept` を試す
3. `EAGAIN` なら poller へ関心を登録する

4. readiness 通知が来たら再開する

この流れが、そのまま `select` / `poll` / `epoll` の世界へ続きます。

> **注意**
> `readable` や `writable` は level で変化します。通知を 1 回受け取ったからといって、後続の `read` / `write` が何度でも成功するわけではありません。1 回進んだあとすぐ `EAGAIN` へ戻ることも普通にあります。

Go から見ると何が起きているか

Go の `net.Conn` は user code からは blocking API に見えます。`Read` を呼べば data が来るまで待ってくれますし、`Write` も普通の関数のように見えます。しかし Go runtime は多数 connection を少数 thread で扱うため、内部では nonblocking socket と poller を使います。

つまり Go は、「nonblocking と poller の面倒を runtime が肩代わりし、その上へ goroutine ごとの blocking 風 API を載せる」設計です。この構造を知っておくと、goroutine ごとの code が簡単に見える理由が分かります。下の層で readiness をまとめて待っているからこそ、上の層では connection ごとに素直な code が書けます。

この章で作りたい感覚

この章の到達点は、次の 3 つの感覚です。

- `EAGAIN` は制御フローの一部である
- readiness は completion ではなく、「今なら少し進める」の通知である
- nonblocking I/O は busy loop ではなく、poller と組み合わせて初めて意味を持つ

次章では、その poller を具体的に見ます。`select` は何がつらいのか。`poll` は何を改善するのか。`epoll` は何を一度登録し、何を返してくるのか。event loop の最小形と一緒に整理します。

`select` / `poll` / `epoll`

nonblocking I/O を導入すると、次に必要になるのは「複数 fd の readiness をまとめて待つ仕組み」です。1 本の socket に対して `EAGAIN` を見ただけでは進みません。どの fd がいつ readable / writable になったかを、1 箇所で待って dispatch する必要があります。この章では、そのための代表的な API である `select`、`poll`、`epoll` を整理します。

> **この章の到達点**
> この章では次を押さえます。
>
> - `select`、`poll`、`epoll` の基本的な違い
> - `epoll` が interest list と ready list をどう扱うか
> - level-triggered と edge-triggered の考え方
> - event loop が「状態機械」になる理由

1 本ずつ見るのではなく、まとめて待つ

nonblocking socket を 1000 本持っているとします。各 socket に対して毎回 `read` を試し、`EAGAIN` なら次へ進む、を愚直に回すだけでは、ほとんどの時間を空振りに使ってしまいます。欲しいのは、「変化があった fd だけ教えてほしい」という仕組みです。

その発想が multiplexing API です。user space は「この fd が readable になるのを待ちたい」「この fd が writable になるのを待ちたい」と登録し、kernel は ready になった fd をまとめて返します。これにより 1 thread でも多くの connection を扱えるようになります。

`select`

`select` は古典的な API です。`fd_set` に監視したい fd を詰め、read / write / exception ごとに集合を渡し、ready になったものだけが返ります。概念は分かりやすいですが、次の弱点があります。

- `fd_set` を毎回作り直す必要がある
- 最大 fd 数に上限がある
- ready でない fd も毎回 scan する

そのため、小さな program の導入としてはよくても、大量 connection を扱う場面では窮屈です。

`poll`

`poll` は `struct pollfd` の配列を渡します。`select` より API は素直で、fd 上限の扱いも改善されます。ただし本質的には「毎回 user space から監視対象を全部渡し、kernel も全部見て、user space も返ってきた配列を全部なめる」構造です。つまり fd 数が多いと scan コストが効いてきます。

`poll` は `select` より書きやすいことが多いですが、監視対象が大きくなったときの構造的な負担はまだ残ります。

`epoll`

Linux で大量 fd を扱うときの中心が `epoll` です。`epoll` の発想は、「毎回全部渡すのではなく、監視対象を kernel 側に保持してもらう」ことです。

典型的な流れは次です。

1. `epoll_create1` で epoll instance を作る
2. `epoll_ctl(ADD)` で fd と監視したい event を登録する
3. `epoll_wait` で ready event を待つ
4. 必要なら `epoll_ctl(MOD/DEL)` で登録内容を更新する

ここで user space は、毎回 1000 本の fd 全部を渡し直しません。kernel 側が interest list を持ち、状態変化があったものを ready list へ載せ、`epoll_wait` で user space に返します。この構造が `epoll` の強みです。

> **編集注記**
> `epoll` は「速い魔法」ではありません。毎回全部をなめる構造を減らし、ready になった fd を中心に扱えるようにする設計です。結局 user space は、返ってきた event に対して state machine を回す必要があります。

最小の `epoll` event loop

C で最小限の骨格だけ書くと、次のようになります。

```
int ep = epoll_create1(0);
set_nonblock(lfd);
```

```
struct epoll_event lev = {
    .events = EPOLLIN,
    .data.fd = lfd,
};
epoll_ctl(ep, EPOLL_CTL_ADD, lfd, &lev);

for (;;) {
    struct epoll_event events[128];
    int n = epoll_wait(ep, events, 128, -1);

    for (int i = 0; i < n; i++) {
        int fd = events[i].data.fd;

        if (fd == lfd) {
            int cfd = accept(lfd, NULL, NULL);
            set_nonblock(cfd);

            struct epoll_event cev = {
                .events = EPOLLIN,
                .data.fd = cfd,
            };
            epoll_ctl(ep, EPOLL_CTL_ADD, cfd, &cev);
            continue;
        }

        ssize_t nr = read(fd, buf, sizeof(buf));
        if (nr > 0) {
            queue_response(fd, buf, (size_t)nr);
        } else if (nr == 0) {
            close(fd);
        } else if (errno != EAGAIN && errno != EWOULDBLOCK) {
            close(fd);
        }
    }
}
```

この code では細かい error handling や write 側の管理を省いていますが、骨格は見えます。`epoll_wait` が ready fd の集合を返し、listening socket なら `accept`、client socket なら `read` を進めます。つまり event loop は「どの fd で、どの状態を、どこまで進めるか」を管理する dispatcher です。

1 event = 1 request 完了ではない

ここで初心者が最も引っかかりやすい点を強調します。`epoll_wait` が 1 回 event を返したからといって、1 request 全体が処理し終わるわけではありません。返ってくるのは readiness です。たとえば `EPOLLIN` が来ても、1 回 `read` しただけでは HTTP header の途中までしか読めないかもしれません。逆に `EPOLLOUT` が来ても、送信 queue を全部出し切れるとは限りません。

そのため event loop は、protocol parser と送信 queue を持つ state machine になります。fd ごとに「header 読み中」「body 読み中」「返信待ち」「送信中」といった application state を持ち、その state に応じて `EPOLLIN` / `EPOLLOUT` の扱いを変えます。ここが thread-per-connection の code より少し難しい点です。

level-triggered と edge-triggered

`epoll` には大きく 2 つの通知モデルがあります。

- level-triggered 条件が満たされている間、待つたびに event が返り得る
- edge-triggered 状態が「変化した瞬間」を中心に event が返る

level-triggered は素直です。まだ読める data が残っているなら、次の `epoll_wait` でも再び `EPOLLIN` が返る可能性があります。したがって導入としては理解しやすいです。

edge-triggered は通知回数を減らせますが、使う側の責務が増えます。event を受け取ったら `read` や `accept` を `EAGAIN` になるまで draining しないと、残っている data に再度気付けない可能性があるからです。したがって ET を使うなら、nonblocking と drain loop を必ずセットで考えます。

```
for (;;) {
    ssize_t nr = read(fd, buf, sizeof(buf));
    if (nr > 0) {
        consume(buf, (size_t)nr);
        continue;
    }
    if (nr == 0) {
        close(fd);
        break;
    }
    if (errno == EAGAIN || errno == EWOULDBLOCK) {
        break;
    }
    close(fd);
    break;
}
```

accept loop も drain する

ET のときは client socket だけでなく listening socket も同じです。`EPOLLIN` が 1 回来たからといって、新規 connection が 1 本だけとは限りません。複数 connection が queue に積まれているかもしれないので、`accept` も `EAGAIN` まで回します。

この点は見落とされやすいですが非常に重要です。ET で `accept` を 1 回だけ呼んで満足すると、queue に残った接続に気付きにくくなります。

`EPOLLOUT` は必要なときだけ

write interest の扱いも event loop の定石があります。常に `EPOLLOUT` を監視していると、多くの socket が「今は少し書ける」状態なので、不要な wakeup が増えます。通常は、未送信 data がある socket にだけ `EPOLLOUT` を付け、全部送り切ったら外します。

この pattern は server の効率に直結します。readable event だけを見ている段階より、ここで初めて user-space buffer 管理が前面に出てきます。

`select` / `poll` / `epoll` の使い分けの感覚

実務では Linux なら `epoll` が中心ですが、学習上は次の感覚が大切です。

- `select` 概念の導入として分かりやすい
- `poll` API は素直だが、毎回全件 scan の構造は残る
- `epoll` Linux で大規模 fd を扱う中心。interest を kernel 側へ保持する

つまり、「新しい API ほど偉い」というより、「監視対象をどこで保持し、毎回何を scan するか」の違いで見ると整理しやすいです。

注意
`epoll` は readiness API であって completion API ではありません。`epoll_wait` が返したこと自体は、送信完了や message 完結を保証しません。この区別を曖昧にすると、short read/write や protocol parser の実装で混乱しやすくなります。

Go runtime への前振り

Go runtime は Linux 上では `epoll` 系の backend を使って network I/O を待ちます。ただし user code へは event loop を直接見せません。goroutine ごとに blocking 風の code を書けるようにしつつ、裏では nonblocking fd と poller を使って待ち合わせを共有します。

したがって C で `epoll` event loop を書くことと、Go で connection ごとに goroutine を立てることは、見た目ほど遠くありません。前者では application が state machine を明示的に持ち、後者では runtime が待ち合わせの一部を肩代わりしています。次章では、その橋をもっと具体的に見ます。

kernel 側の待ち合わせと runtime への橋

ここまでで user space の API はだいぶ見えました。この章では少し視点を下げ、`listen` backlog、socket buffer、sleep/wakeup、TCP/UDP の違いを最小限だけ押さえたうえで、Go runtime の `netpoll` へ戻ります。狙いは kernel source の完全読解ではありません。user space の `read` / `write` / `epoll_wait` が、kernel 側のどの状態変化と対応しているかを結び付けることです。

この章の到達点
この章では次を押さえます。

- `listen` backlog をどう理解するとよいか
- send buffer / receive buffer と backpressure のつながり
- loopback と TCP/UDP の違いを最小限どう捉えるか
- readiness 通知が goroutine の park/unpark とどうつながるか

`listen` backlog の最小像

`listen(fd, backlog)` を呼ぶと、server socket は passive open の状態へ入り、新しい接続を受ける入口になります。学習の最初の理解としては、「`accept` 待ちの接続をためる queue の大きさに関わる」と捉えるのがよいです。つまり application が `accept` で取り出すより速く接続要求が来ると、その queue に積まれ、あふれると失敗や再試行が起こり得ます。

実際の Linux TCP stack はもう少し複雑です。handshake の途中段階と、`accept` 可能になった接続の扱いは分かれており、sysctl tuning も関わります。しかし user space の server 設計として最初に大事なのは、「`accept` を遅らせると queue が詰まる」「backlog は無限ではない」という感覚です。

つまり `accept` loop の設計は単なる書き方の違いではありません。遅い `accept` は新規接続の入口そのものに影響します。event loop で listening socket を優先的にさばく理由の 1 つはここにあります。

socket buffer と data の流れ

connected TCP socket には送受信 buffer があります。大まかには次のように見れば十分です。

1. 相手から届いた byte 列は receive buffer へ入る
2. user space の `read` が receive buffer から取り出す
3. user space の `write` は send buffer へ byte 列を積む
4. kernel が send buffer から network へ送り出す

このモデルを持つと、`readable` / `writable` の意味が分かりやすくなります。receive buffer に data があれば readable です。send buffer に空きがあれば writable です。逆に receive buffer が空なら `read` は進まず、send buffer が詰まっていれば `write` は進みにくくなります。

backpressure はどこから来るか

backpressure は、要するに「下流が遅いので上流も進みにくくなる」現象です。network I/O では、peer が読まない、回線が遅い、receiver 側 application が詰まる、といった理由で発生します。user space から見ると、その結果は short write や `EAGAIN` として現れます。

このとき重要なのは、`write` が詰まった理由を「kernel が意地悪している」ように見ないことです。実際には、相手がまだ受け取れていないから先へ進めないだけです。network I/O は local な function call に見えても、本質的には遠隔の相手との同期です。backpressure を理解すると、送信 queue と timeout の必要性が腹落ちしやすくなります。

loopback は特別か

手元の学習では `127.0.0.1` や `::1` をよく使います。loopback は NIC を経由しないので、外の network よりずっと単純で速いです。ただし socket としての意味が変わるわけではありません。listening socket を作り、connect し、send/receive buffer を使い、`epoll` で readiness を待つ、という大きな構造は同じです。

したがって学習初期には loopback で十分です。packet loss や MTU や実 NIC の細部を無視しやすく、I/O 待ちの骨格だけに集中できます。本書もその前提で話を進めます。

TCP と UDP の違いを最小限だけ

本書の主線は TCP ですが、UDP との違いを最小限だけ押さえておくと整理しやすいです。

- TCP connection 指向。順序があり、byte stream として見える。message 境界は保持しない
- UDP connectionless。datagram 単位で見え、message 境界が残る。到達保証や順序保証はない

ただし readiness という観点では共通点もあります。どちらも「今なら `recv` 系が進みそう」「今なら `send` 系が進みそう」を待ちます。違うのは data の見え方です。TCP は stream なので parser が境界を復元しなければいけません。UDP は 1 datagram が 1 単位として見えます。

data 到着から wakeup までの最小像

細部をかなり省略して、data 到着から user space 再開までを最小限の段階で書くと次のようになります。

1. peer から packet が届く
2. kernel network stack がそれを該当 socket へ結び付ける
3. socket receive buffer に読める data が載る
4. その socket を待っている thread や poller が wakeup 対象になる
5. `epoll_wait` あるいは blocked な読み手が再開する
6. user space が `read` して data を取り出す

この流れで大事なのは、wake up の基準が「application message 完成」ではなく、socket の状態変化だということです。たとえば 1 byte だけ届いても wakeup され得ます。逆に protocol 的には 1 request ぶん揃っていても、user space が parser を回さない限りそれは単なる byte 列です。

> **編集注記**
> readiness と protocol completion を分けて考える癖はとても重要です。kernel が知っているのは通常、socket buffer と error/EOF の状態までです。HTTP request が完成したか、独自 protocol の frame が揃ったかは、多くの場合 user space parser の責務です。

sleep/wakeup と scheduler

`syscalls-process-primer` で見た sleep/wakeup の感覚は、ここでもそのまま使えます。blocking `read` は「この socket が readable になるまで寝る」ですし、`epoll_wait` は「登録した fd のどれかに興味ある event が起きるまで寝る」です。違うのは待ち対象の粒度だけです。

この観点で見ると、event loop も結局は scheduler の 1 種に見えます。ready になった fd を取り出し、どの connection state をどこまで進めるかを決めるからです。thread scheduler が runnable thread を選ぶのに対し、event loop は ready fd とその state を選びます。Go runtime の `netpoll` を理解しやすくする鍵もここにあります。

Go runtime の `netpoll`

Go では `net.Conn.Read` や `net.Conn.Write` が blocking API のように見えます。しかし runtime は多数 goroutine を少数 thread で回したいので、内部では nonblocking fd と poller を使います。Linux ではその backend の中心が `epoll` です。

概念的には、次のような流れだと考えると分かりやすいです。

1. network fd は nonblocking として扱われる
2. goroutine が `Read` / `Write` を試す
3. すぐ進めないなら runtime はその goroutine を park する
4. runtime の poller は fd readiness を待つ
5. ready になったら対応する goroutine を runnable に戻す
6. その goroutine が再び CPU を得て処理を続ける

ここで重要なのは、「goroutine が待っている」と「OS thread がそのまま塞がっている」を分けることです。runtime は goroutine を park/unpark するので、I/O 待ちの間も他の goroutine を進められます。これが goroutine-per-connection が比較的自然に書ける理由です。

C の event loop と Go の goroutine-per-connection

C で `epoll` を直接使うと、application 自身が fd ごとの state machine をはっきり書くことになります。どの fd が readable か、どこまで parse したか、未送信 data がどれだけ残っているかを明示的に管理します。

Go では見た目がかなり違います。connection ごとに goroutine を作り、`Read` が返るまでその goroutine の実行を止めればよいので、application code は直線的です。ただし state machine が消えたわけではありません。待ち合わせの一部を runtime が肩代わりし、goroutine scheduler と `netpoll` の協調で隠れているだけです。

この比較を持っておくと、Go の code が「簡単だから kernel の都合を無視できる」のではなく、「kernel の readiness を runtime が受け止める設計になっている」から簡単に見えるのだと分かります。

Go の最小 echo server をもう一度見る

たとえば次のような Go code はとても素直です。

```
ln, err := net.Listen("tcp", ":8080")
if err != nil {
    log.Fatal(err)
}
for {
    conn, err := ln.Accept()
    if err != nil {
        continue
    }
    go func(c net.Conn) {
        defer c.Close()
        buf := make([]byte, 4096)
        for {
            n, err := c.Read(buf)
            if n > 0 {
                if _, werr := c.Write(buf[:n]); werr != nil {
                    return
                }
            }
            if err != nil {
                return
            }
        }
    }(conn)
}
```

application 側は connection ごとに素直な loop を書けます。しかし runtime の下では、I/O 待ちの goroutine を park し、fd readiness で再開させる仕組みが動いています。つまり user code の見た目は thread-per-connection に近いのに、実際の待ち合わせは readiness-based multiplexing に支えられているわけです。

readiness ベースの待ち合わせの限界と次の話題

ここまでのモデルは非常に強力ですが、万能ではありません。大量の状態管理、送信 queue、timer 管理、protocol parser、fairness といった問題は依然として残ります。また Linux には completion-based の色が強い `io_uring` もあります。ただし、それらへ進む前に readiness ベースの待ち合わせをしっかり理解しておく価値は大きいです。

なぜなら、`epoll` と `netpoll` の世界を理解しておくと、「completion とは何が違うのか」を比較できるからです。基準点がないまま新しい API を見るより、既存の待ち合わせモデルを一度手に入れたほうが学習しやすいです。

> **注意**
> Go の network I/O を「goroutine だから thread を使わない」と表現するのは正確ではありません。実際には OS thread も poller thread も使います。重要なのは、goroutine の待ちと thread の占有を runtime がうまく分離していることです。

この章で作りたい感覚

この章の到達点は次です。

- backlog は接続待ちの入口に関わる
- send/receive buffer は readable / writable と backpressure の基礎である
- wakeup は socket 状態変化に基づく
- Go の `netpoll` は kernel readiness と goroutine scheduler の橋である

ここまで来ると、`syscalls-process-primer` の sleep/wakeup と `go-runtime-primer` の `netpoll` がかなり近い話に見えてきます。最後の章では、復習、観測コマンド、よくあるつまずき、演習、次に読む source をまとめます。

付録と次の一歩

本編では、`socket`、blocking / nonblocking I/O、`select` / `poll` / `epoll`、socket buffer、Go runtime の `netpoll` を一本の線で整理しました。この章では、復習、観測コマンド、用語、よくあるつまずき、短い演習、次に読む資料をまとめます。新しい topic を増やすというより、「ここまでの理解を手元で確かめ、次へ伸ばす」ための章です。

この章の到達点

この章では次を押さえます。

- 各章の復習ポイント
- 手元で観測するときの最小コマンド群
- よく混乱する用語と誤解
- 次に読むとよい manual / source / 文書

章ごとの復習ポイント

ここまでの内容を短く戻すと、各章の核は次のように整理できます。

- 導入: network I/O は protocol の話であるだけでなく、待ち合わせと再開の話でもある
- blocking I/O: listening socket と connected socket を分け、`accept` / `read` / `write` が sleep し得ることを見る
- nonblocking: `O_NONBLOCK` と `EAGAIN` により、待ちを user space 側へ戻す
- multiplexing: `select` / `poll` / `epoll` は複数 fd の readiness をまとめて待つ
- kernel bridge: backlog、buffer、backpressure、wake up、`netpoll` が一続きで見える

この順序を持っているだけで、network I/O の学習はかなり迷いにくくなります。`epoll` の man page を読んでも、「これは readiness の登録 API だ」「completion を返すわけではない」と整理しやすくなります。

観測コマンドの導線

理解を定着させるには、短い program を動かして system call と socket 状態を見るのが有効です。最小の導線としては次が便利です。

```
$ strace -f -tt -e trace=network,epoll_ctl,epoll_wait,read,write ./server
$ ss -ntlp | grep 8080
$ lsof -iTCP:8080 -n -P
$ tcpdump -i lo tcp port 8080
$ go test -trace trace.out ./...
$ go tool trace trace.out
```

`strace` は「どの syscall で待っているか」を見る入口です。blocking server なら `accept` や `read` で止まり、`epoll` server なら `epoll_wait` を中心に見えるはずです。`ss` や `lsof` は listening 状態や established connection を観察するのに役立ちます。`tcpdump` は loopback 上でも packet の往復を見せてくれるので、`read` が返る前に何が飛んでいるかの感覚を持ちやすくなります。Go 側は `go tool trace` が、goroutine の block/unblock と scheduler の動きを見る入口になります。

小さな観測課題

次の順で短い観測課題を試すと、本編の概念がかなり手元に定着します。

1. blocking echo server を作り、1 client が黙っていると他が止まることを確認する
2. 同じ server を thread-per-connection にして、単純だが thread 数が増えることを確認する
3. socket を nonblocking にし、`EAGAIN` が普通に返ることを観察する
4. `poll` あるいは `epoll` 版へ書き換え、待ちが `epoll_wait` へ集約されることを `strace` で見る
5. Go 版 echo server を `trace` し、goroutine が I/O 待ちで park/unpark される様子を眺める

ここで大切なのは、大きな benchmark を走らせることではありません。短い program で「いま何を見たいのか」をはっきりさせることです。観測結果を source や manual のどこへ戻すかが分かれば十分です。

用語小事典

- file descriptor user space から I/O 対象を指す整数ハンドル
- listening socket 新しい接続を受け取る入口になる socket
- connected socket 接続後の実データ送受信用 socket
- blocking I/O 条件が整うまで caller を sleep させる I/O
- nonblocking I/O 進めないときは sleep せず、`EAGAIN` などで caller へ戻す I/O
- readiness 「今なら少なくとも 1 回は block せず進めそう」という状態
- backpressure 下流の遅さが上流の進みを抑える現象
- backlog `listen` 側の接続待ち capacity に関わる量
- level-triggered 条件が成立している間は event が返り続け得るモデル
- edge-triggered 状態変化の瞬間を中心に event を返すモデル
- `netpoll` Go runtime が network fd readiness と goroutine scheduler を結ぶ仕組み

よくあるつまずき

`readable` なら 1 request 全体を読めると思ってしまう

そうではありません。`readable` は socket buffer の状態変化を示すだけです。TCP は byte stream なので、application message の境界は自分で復元する必要があります。

nonblocking は busy loop だと思ってしまう

違います。正しい nonblocking I/O は poller と組み合わせます。`EAGAIN` が返ったら、何度も空打ちするのではなく、ready になるまで別の仕事へ移るのが基本です。

`epoll` があれば state machine は不要だと思ってしまう

不要にはなりません。`epoll` は ready fd を返すだけです。どこまで parse 済みか、返信が何 byte 残っているか、timeout をどう扱うかは application state の責務です。

Go では `epoll` を意識しなくてよいと思ってしまう

普段の application code では直接触れなくてよい場面が多いですが、性能問題や block の原因を理解するときには土台のモデルが効きます。runtime が readiness を隠してくれているだけで、待ち合わせ自体が消えるわけではありません。

`writable` なら全部送れると思ってしまう

これも違います。send buffer の空きは有限です。大量送信では short write や `EAGAIN` が普通に起きるので、未送信 data の管理が必要です。

> **注意**
> `epoll` を使うだけで program が自動的に速くなるわけではありません。application protocol が重い、送信 queue が巨大、timer 管理が雑、lock contention が大きい、といった別の要因はそのまま残ります。

短い演習案

- 演習 1: blocking echo server を書き、`nc` を 2 つ開いて片方を黙らせたときの挙動を説明する
- 演習 2: 同じ server を nonblocking + `poll` で書き直し、`EAGAIN` の出方を確認する
- 演習 3: `epoll` server で `EPOLLOUT` を常時監視した版と、未送信 data があるときだけ監視する版を比較する
- 演習 4: Go で connection ごとに goroutine を立てる server を書き、`go tool trace` で block/unblock を観察する
- 演習 5: TCP と UDP の echo server を両方書き、message 境界の違いを説明する

これらの演習は「全部完成させる」ことより、「何が難しいのか」を言葉にできることが大切です。たとえば `epoll` server が難しいなら、parser と送信 queue が必要になる点を説明できれば十分に前進しています。

`syscalls-process-primer` と `go-runtime-primer` へ戻る読み方

本書を読み終えたら、既存シリーズへ戻ると見え方が変わります。

まず `syscalls-process-primer` へ戻ると、sleep/wakeup、scheduler、wait queue の説明が network I/O と直結して見えます。`read` が止まること、`epoll_wait` がまとめて待つこと、wake up が条件変化で起きることが、抽象論ではなく具体例になります。

次に `go-runtime-primer` へ戻ると、`netpoll` が突然出てくる謎の部品ではなくなります。Go runtime は kernel readiness を受け取り、goroutine を park/unpark する橋として `netpoll` を使っています。つまり本書の知識は、そのまま runtime 読解の前提になります。

次に読むなら

本書の次に進む方向は大きく 4 つあります。

- protocol を深める: TCP の輻輳制御、再送、flow control、HTTP/TLS へ進む
- kernel を深める: Linux network stack、socket internals、NAPI、driver 方向へ進む
- runtime を深める: Go の `netpoll` 実装、scheduler、timer との協調を source で追う
- API を広げる: `io_uring`、`kqueue`、IOCP など readiness/completion の比較へ進む

最初の一冊で全部へ行く必要はありません。むしろ「待つとは何か」「ready とは何か」を 1 本持っておくと、どの枝へ行っても学習しやすくなります。

参考資料

- `socket(2)`
- `socket(7)`
- `tcp(7)`
- `udp(7)`
- `select(2)`
- `poll(2)`
- `epoll(7)`
- `strace(1)`
- `ss(8)`
- Go runtime の `netpoll` 関連 source
- `go tool trace`
- `The Linux Programming Interface`
- `UNIX Network Programming`

> **編集注記**
> network I/O は一見すると API の数が多く、protocol の細部も多いので散らばって見えます。しかし中心にある問いは比較的単純です。今は進めるのか、待つべきなのか、誰が起こすのか。本書はその 3 つの問いを揃えることを目指しました。

おわりに

network I/O を理解するために、最初から巨大な protocol 図や kernel source 全体図が必要なわけではありません。まずは `socket` を作り、block し、nonblocking にし、ready を待ち、再開する、という一連の流れを掴むことが大切です。その流れが見えると、`epoll` も Go の `netpoll` も急に別世界の技術ではなくなります。

一度この地図を持つと、C の event loop を書くときも、Go の goroutine server を読むときも、「いまは待ち合わせのどの層を見ているのか」を言葉にしやすくなります。そこまで来れば、次は protocol、kernel、runtime のどの方向へ進んでも、理解を積み上げやすくなります。